AGLC CRM Deployment Files Distribution

WhatsApp & Telegram Integration Guide

Date: March 13, 2026
Purpose: Share deployment files and commands via messaging platforms
Status: Ready to Deploy

PART 1: PREPARATION - CREATE SHAREABLE FILES

1.1 Create Condensed Deployment Script

```
# Create a single, condensed deployment script
cat > ~/AGLC_CRM_DEPLOY.sh << 'EOF'
#!/bin/bash
# AGLC CRM One-Click Deployment Script
# Copy and paste this entire script into your terminal

set -e

echo "🚀 AGLC CRM Deployment Starting..."

# Colors
GREEN='\033[0;32m'
BLUE='\033[0;34m'
RED='\033[0;31m'
NC='\033[0m'

# 1. Install prerequisites
echo -e "${BLUE}Installing prerequisites...${NC}"
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER
newgrp docker

# 2. Clone repositories
echo -e "${BLUE}Cloning repositories...${NC}"
mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm

# 3. Create environment file
echo -e "${BLUE}Creating environment configuration...${NC}"
cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
```

```
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF

# 4. Create docker-compose.yml
echo -e "${BLUE}Creating Docker configuration...${NC}"
cat > docker-compose.yml << 'DOCKEREOF'
version: '3.8'
services:
  postgres:
    image: postgres:15-alpine
    container_name: aglc-crm-postgres
    environment:
      POSTGRES_USER: aglc_user
      POSTGRES_PASSWORD: aglc_password_dev
      POSTGRES_DB: aglc_crm_dev
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U aglc_user"]
      interval: 10s
      timeout: 5s
      retries: 5
  pgadmin:
    image: dpage/pgadmin4:latest
    container_name: aglc-crm-pgadmin
    environment:
      PGADMIN_DEFAULT_EMAIL: admin@aglc.com
      PGADMIN_DEFAULT_PASSWORD: admin_password_dev
    ports:
      - "5050:80"
    depends_on:
      - postgres
volumes:
  postgres_data:
networks:
  default:
    name: aglc-crm-network
DOCKEREOF

# 5. Start Docker services
echo -e "${BLUE}Starting Docker containers...${NC}"
```

```
docker-compose up -d
sleep 5

# 6. Install dependencies
echo -e "${BLUE}Installing Node.js dependencies...${NC}"
pnpm install

# 7. Run database migrations
echo -e "${BLUE}Running database migrations...${NC}"
pnpm db:push

# 8. Build project
echo -e "${BLUE}Building project...${NC}"
pnpm build

echo ""
echo -e "${GREEN}✅ Deployment Complete!${NC}"
echo ""
echo "📍 Access Points:"
echo "  - Frontend: http://localhost:5173"
echo "  - Backend API: http://localhost:3000"
echo "  - Database: localhost:5432"
echo "  - pgAdmin: http://localhost:5050"
echo ""
echo "🚀 Start development server:"
echo "  cd ~/projects/aglc/aglc-crm"
echo "  pnpm dev"
echo ""

EOF

chmod +x ~/AGLC_CRM_DEPLOY.sh
```

1.2 Create Quick Reference Card

```
# Create a quick reference card
cat > ~/AGLC_CRM_QUICK_REFERENCE.txt << 'EOF'
╔═══════════════════════════════════════════════════════════════╗
║          AGLC CRM - QUICK REFERENCE & COMMANDS                ║
╚═══════════════════════════════════════════════════════════════╝

📋 ONE-CLICK DEPLOYMENT
────────────────────────────────────────────────────────────────
bash ~/AGLC_CRM_DEPLOY.sh

⚡ QUICK START (After deployment)
────────────────────────────────────────────────────────────────
cd ~/projects/aglc/aglc-crm
pnpm dev

🌐 ACCESS POINTS
────────────────────────────────────────────────────────────────
Frontend:     http://localhost:5173
Backend API:  http://localhost:3000
pgAdmin:      http://localhost:5050
Database:     localhost:5432

🛑 STOP SERVICES
────────────────────────────────────────────────────────────────
cd ~/projects/aglc/aglc-crm
docker-compose down

🔄 RESTART SERVICES
────────────────────────────────────────────────────────────────
cd ~/projects/aglc/aglc-crm
docker-compose restart

📊 USEFUL COMMANDS
────────────────────────────────────────────────────────────────
Check status:        docker-compose ps
View logs:           docker-compose logs -f postgres
Database CLI:        docker-compose exec postgres psql -U aglc_user -d
aglc_crm_dev
Run tests:           pnpm test
Format code:         pnpm format
Type check:          pnpm check

🔐 DATABASE CREDENTIALS
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
User:     aglc_user
Password: aglc_password_dev
Database: aglc_crm_dev
Host:     localhost
Port:     5432

🔐 PGADMIN CREDENTIALS
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
Email:    admin@aglc.com
Password: admin_password_dev
URL:      http://localhost:5050

⚠️  TROUBLESHOOTING
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
Port in use:         lsof -ti:3000 | xargs kill -9
Clear cache:         pnpm store prune
Reset database:      docker-compose down -v && docker-compose up -d
Full reset:          bash ~/AGLC_CRM_RESET.sh

📞 SUPPORT
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━
Documentation:  See AGLC_CRM_LOCAL_DEPLOYMENT_COMMANDS.pdf
GitHub:         https://github.com/aglc/aglc-crm
Issues:         https://github.com/aglc/aglc-crm/issues

EOF

cat ~/AGLC_CRM_QUICK_REFERENCE.txt
```

PART 2: WHATSAPP DISTRIBUTION

2.1 WhatsApp Web Method (Manual)

Step 1: Prepare Files

```
# Create a text file with deployment commands
cat > ~/AGLC_DEPLOY_COMMANDS.txt << 'EOF'
AGLC CRM DEPLOYMENT COMMANDS
============================

Copy and paste these commands one by one into your terminal:

1️⃣ INSTALL PREREQUISITES
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

2️⃣ CLONE REPOSITORIES
mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm

3️⃣ CREATE ENVIRONMENT FILE
cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF

4️⃣ START SERVICES
docker-compose up -d
pnpm install
pnpm db:push

5️⃣ RUN DEVELOPMENT SERVER
pnpm dev

Access: http://localhost:5173

EOF
```

```
cat ~/AGLC_DEPLOY_COMMANDS.txt
```

Step 2: Share via WhatsApp Web

1. Open WhatsApp Web (web.whatsapp.com)
2. Open the chat or group where you want to share
3. Copy the commands from the text file
4. Paste into WhatsApp message
5. Send

2.2 WhatsApp API Integration (Automated)

```
# Install Twilio CLI (for WhatsApp API)
npm install -g twilio-cli

# Authenticate with Twilio
twilio login

# Create WhatsApp message script
cat > ~/send_whatsapp.js << 'EOF'
const twilio = require('twilio');

const accountSid = process.env.TWILIO_ACCOUNT_SID;
const authToken = process.env.TWILIO_AUTH_TOKEN;
const client = twilio(accountSid, authToken);

const deploymentMessage = `
🚀 AGLC CRM DEPLOYMENT

Here are your deployment commands:

1. Install prerequisites:
sudo apt update && sudo apt upgrade -y

2. Clone repositories:
mkdir -p ~/projects/aglc && cd ~/projects/aglc
gh repo clone aglc-crm

3. Setup project:
cd aglc-crm
docker-compose up -d
pnpm install
pnpm db:push

4. Start development:
pnpm dev

Access: http://localhost:5173

📄 Full guide: See attached PDF file
`;

client.messages
  .create({
    from: 'whatsapp:+1234567890',  // Your Twilio WhatsApp number
```

```
    to: 'whatsapp:+966XXXXXXXXX',   // Recipient's WhatsApp number
    body: deploymentMessage
  })
  .then(message => console.log(`Message sent: ${message.sid}`))
  .catch(error => console.error(`Error: ${error.message}`));

EOF

node ~/send_whatsapp.js
```

PART 3: TELEGRAM DISTRIBUTION

3.1 Telegram Bot Method

Step 1: Create Telegram Bot

```
# 1. Open Telegram and search for @BotFather
# 2. Send /start
# 3. Send /newbot
# 4. Follow the prompts to create a new bot
# 5. Copy your bot token (looks like: 123456:ABC-DEF1234ghIkl-
zyx57W2v1u123ew11)

# Set bot token as environment variable
export TELEGRAM_BOT_TOKEN="YOUR_BOT_TOKEN_HERE"
```

Step 2: Create Telegram Bot Script

```
cat > ~/send_telegram.js << 'EOF'
const axios = require('axios');

const botToken = process.env.TELEGRAM_BOT_TOKEN;
const chatId = process.env.TELEGRAM_CHAT_ID;  // Your Telegram chat ID

const deploymentMessage = `
🚀 <b>AGLC CRM DEPLOYMENT GUIDE</b>

<b>Step 1: Install Prerequisites</b>
<code>sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh</code>

<b>Step 2: Clone Repositories</b>
<code>mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm</code>

<b>Step 3: Create Environment File</b>
<code>cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF</code>

<b>Step 4: Start Services</b>
<code>docker-compose up -d
pnpm install
pnpm db:push</code>

<b>Step 5: Run Development Server</b>
<code>pnpm dev</code>

<b>Access Points:</b>
• Frontend: http://localhost:5173
• Backend API: http://localhost:3000
```

```
• pgAdmin: http://localhost:5050

📄 Full documentation in PDF file
`;

const url = `https://api.telegram.org/bot${botToken}/sendMessage`;

axios.post(url, {
  chat_id: chatId,
  text: deploymentMessage,
  parse_mode: 'HTML'
})
.then(response => console.log('Message sent successfully'))
.catch(error => console.error('Error sending message:', error.message));

EOF

npm install axios
node ~/send_telegram.js
```

3.2 Telegram Channel Distribution

Step 1: Create Telegram Channel

1. Open Telegram
2. Click “+” button
3. Select “New Channel”
4. Name it “AGLC CRM Deployment”
5. Set to Public
6. Add description

Step 2: Post Deployment Guide

```
# Create a comprehensive Telegram post
cat > ~/telegram_post.txt << 'EOF'
🚀 AGLC CRM DEPLOYMENT GUIDE

📌 QUICK START (One Command)
bash ~/AGLC_CRM_DEPLOY.sh

📋 STEP-BY-STEP INSTRUCTIONS

1️⃣ Install Prerequisites (5 minutes)
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo usermod -aG docker $USER

2️⃣ Clone Repositories (2 minutes)
mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm

3️⃣ Setup Environment (1 minute)
cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF

4️⃣ Start Services (5 minutes)
docker-compose up -d
sleep 5
pnpm install
pnpm db:push

5️⃣ Run Development Server (1 minute)
pnpm dev

✅ VERIFICATION
```

```
Open in browser:
- Frontend: http://localhost:5173
- Backend: http://localhost:3000
- pgAdmin: http://localhost:5050

🛑 STOP SERVICES
docker-compose down

📞 SUPPORT
See attached PDF for full documentation

EOF

cat ~/telegram_post.txt
```

PART 4: EMAIL DISTRIBUTION

4.1 Create Email-Friendly Format

```
# Create email-friendly deployment guide
cat > ~/AGLC_CRM_EMAIL_GUIDE.html << 'EOF'
<!DOCTYPE html>
<html>
<head>
    <style>
        body { font-family: Arial, sans-serif; line-height: 1.6; color:
#333; }
        .container { max-width: 600px; margin: 0 auto; padding: 20px; }
        h1 { color: #0052CC; border-bottom: 3px solid #0052CC; padding-
bottom: 10px; }
        h2 { color: #1E40AF; margin-top: 20px; }
        .code-block { background-color: #f4f4f4; border-left: 4px solid
#0052CC; padding: 10px; margin: 10px 0; font-family: monospace; overflow-x:
auto; }
        .step { background-color: #f9f9f9; padding: 15px; margin: 10px 0;
border-radius: 5px; }
        .success { color: #10B981; font-weight: bold; }
        .warning { color: #F59E0B; font-weight: bold; }
        table { width: 100%; border-collapse: collapse; margin: 10px 0; }
        th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
        th { background-color: #0052CC; color: white; }
    </style>
</head>
<body>
    <div class="container">
        <h1>🚀 AGLC CRM Deployment Guide</h1>

        <h2>Quick Start (One Command)</h2>
        <div class="code-block">bash ~/AGLC_CRM_DEPLOY.sh</div>

        <h2>Step-by-Step Instructions</h2>

        <div class="step">
            <h3>Step 1: Install Prerequisites (5 minutes)</h3>
            <div class="code-block">
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
```

```
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
            </div>
        </div>

        <div class="step">
            <h3>Step 2: Clone Repositories (2 minutes)</h3>
            <div class="code-block">
mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm
            </div>
        </div>

        <div class="step">
            <h3>Step 3: Setup Environment (1 minute)</h3>
            <div class="code-block">
cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF
            </div>
        </div>

        <div class="step">
            <h3>Step 4: Start Services (5 minutes)</h3>
            <div class="code-block">
docker-compose up -d
sleep 5
pnpm install
pnpm db:push
            </div>
        </div>

        <div class="step">
            <h3>Step 5: Run Development Server (1 minute)</h3>
            <div class="code-block">pnpm dev</div>
        </div>

        <h2>✅ Verification</h2>
```

```
        <table>
            <tr>
                <th>Service</th>
                <th>URL</th>
                <th>Purpose</th>
            </tr>
            <tr>
                <td>Frontend</td>
                <td>http://localhost:5173</td>
                <td>React application</td>
            </tr>
            <tr>
                <td>Backend API</td>
                <td>http://localhost:3000</td>
                <td>tRPC API server</td>
            </tr>
            <tr>
                <td>pgAdmin</td>
                <td>http://localhost:5050</td>
                <td>Database management</td>
            </tr>
            <tr>
                <td>Database</td>
                <td>localhost:5432</td>
                <td>PostgreSQL</td>
            </tr>
        </table>

        <h2>🛑 Stop Services</h2>
        <div class="code-block">docker-compose down</div>

        <h2>📞 Support</h2>
        <p>For full documentation, see the attached PDF file:
<strong>AGLC_CRM_LOCAL_DEPLOYMENT_COMMANDS.pdf</strong></p>

        <p class="success">✓ Estimated deployment time: 15-20 minutes</p>
    </div>
</body>
</html>

EOF

echo "Email guide created: ~/AGLC_CRM_EMAIL_GUIDE.html"
```

PART 5: GITHUB GIST DISTRIBUTION

5.1 Create GitHub Gist

```
# Create a GitHub Gist with deployment commands
cat > ~/deployment_gist.md << 'EOF'
# AGLC CRM Deployment Guide

## Quick Start

```bash
bash ~/AGLC_CRM_DEPLOY.sh
```
```

Step-by-Step

1. Install Prerequisites

```
sudo apt update && sudo apt upgrade -y
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs postgresql-client git
sudo npm install -g pnpm@10.4.1
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
```

2. Clone Repositories

```
mkdir -p ~/projects/aglc
cd ~/projects/aglc
gh repo clone aglc-crm
cd aglc-crm
```

3. Setup Environment

```
cat > .env.local << 'ENVEOF'
DATABASE_URL=postgresql://aglc_user:aglc_password_dev@localhost:5432/aglc_crm_
VITE_APP_ID=your_app_id
OAUTH_SERVER_URL=https://api.manus.im
VITE_OAUTH_PORTAL_URL=https://portal.manus.im
JWT_SECRET=your_jwt_secret_min_32_chars
NODE_ENV=development
PORT=3000
ENVEOF
```

4. Start Services

```
docker-compose up -d
pnpm install
pnpm db:push
```

5. Run Development Server

```
pnpm dev
```

Access Points

- Frontend: http://localhost:5173
- Backend API: http://localhost:3000
- pgAdmin: http://localhost:5050

Support

See full documentation in PDF file.

EOF

Create the gist

gh gist create ~/deployment_gist.md –public –description “AGLC CRM Deployment Guide”

```
---

## PART 6: SLACK DISTRIBUTION

### 6.1 Post to Slack Channel

```bash
# Create Slack message
cat > ~/slack_message.json << 'EOF'
{
  "text": "🚀 AGLC CRM Deployment Guide",
  "blocks": [
    {
      "type": "header",
      "text": {
        "type": "plain_text",
        "text": "🚀 AGLC CRM Deployment"
      }
    },
    {
      "type": "section",
      "text": {
        "type": "mrkdwn",
        "text": "*Quick Start*\n```bash\nbash ~/AGLC_CRM_DEPLOY.sh\n```"
      }
    },
    {
      "type": "section",
      "text": {
        "type": "mrkdwn",
        "text": "*Access Points*\n• Frontend: http://localhost:5173\n•
Backend: http://localhost:3000\n• pgAdmin: http://localhost:5050"
      }
    },
    {
      "type": "section",
      "text": {
        "type": "mrkdwn",
        "text": "*Full Documentation*\nSee attached PDF file"
      }
    }
  ]
}
EOF
```
```

```
# Send to Slack (requires SLACK_WEBHOOK_URL)
curl -X POST -H 'Content-type: application/json' \
  --data @~/slack_message.json \
  $SLACK_WEBHOOK_URL
```

PART 7: AUTOMATED DISTRIBUTION SCRIPT

7.1 Master Distribution Script

```
# Create master distribution script
cat > ~/distribute_deployment.sh << 'EOF'
#!/bin/bash

set -e

echo "🚀 AGLC CRM Deployment Files Distribution"
echo "========================================="

# Colors
GREEN='\033[0;32m'
BLUE='\033[0;34m'
NC='\033[0m'

# Create distribution directory
mkdir -p ~/aglc_distribution
cd ~/aglc_distribution

echo -e "${BLUE}Creating distribution files...${NC}"

# 1. Copy main deployment script
cp ~/AGLC_CRM_DEPLOY.sh ./

# 2. Copy quick reference
cp ~/AGLC_CRM_QUICK_REFERENCE.txt ./

# 3. Copy deployment commands
cp ~/AGLC_DEPLOY_COMMANDS.txt ./

# 4. Copy PDF guides
cp ~/AGLC_CRM_LOCAL_DEPLOYMENT_COMMANDS.pdf ./
cp ~/AGLC_CRM_GITHUB_NOTION_SETUP.pdf ./

# 5. Create README
cat > README.md << 'READMEEOF'
# AGLC CRM Deployment Files

This folder contains all files needed to deploy the AGLC CRM system.

## Files Included
```

```
1. **AGLC_CRM_DEPLOY.sh** - One-click deployment script
2. **AGLC_CRM_QUICK_REFERENCE.txt** - Quick reference card
3. **AGLC_DEPLOY_COMMANDS.txt** - Step-by-step commands
4. **AGLC_CRM_LOCAL_DEPLOYMENT_COMMANDS.pdf** - Full deployment guide
5. **AGLC_CRM_GITHUB_NOTION_SETUP.pdf** - GitHub & Notion setup guide

## Quick Start

```bash
bash AGLC_CRM_DEPLOY.sh
```
```

Support

See PDF files for complete documentation.

READMEEOF

echo -e "$GREEN✓ Distribution files created$ {NC}" echo "" echo "📁 Location: ~/aglc_distribution" echo "" echo "📤 Share these files via:" echo "• WhatsApp" echo "• Telegram" echo "• Email" echo "• Slack" echo "• GitHub Gist" echo "" echo "✅ Ready for distribution!"

EOF

chmod +x ~/distribute_deployment.sh bash $ /distribute_deployment.sh

---

PART 8: INSTALLATION VERIFICATION SCRIPT

8.1 Create Verification Script

```bash
# Create verification script to check installation
cat > ~/verify_installation.sh << 'EOF'
#!/bin/bash

echo "🔍 AGLC CRM Installation Verification"
echo "====================================="

# Colors
GREEN='\033[0;32m'
RED='\033[0;31m'
YELLOW='\033[1;33m'
NC='\033[0m'

check_command() {
    if command -v $1 &> /dev/null; then
        echo -e "${GREEN}✓${NC} $1 is installed"
        $1 --version 2>/dev/null | head -n 1
    else
        echo -e "${RED}✗${NC} $1 is NOT installed"
    fi
}

echo ""
echo "Checking prerequisites..."
check_command node
check_command pnpm
check_command docker
check_command git
check_command psql

echo ""
echo "Checking Docker services..."
if docker-compose ps &> /dev/null; then
    echo -e "${GREEN}✓${NC} Docker Compose is working"
    docker-compose ps
else
    echo -e "${RED}✗${NC} Docker Compose is not working"
```

```
fi

echo ""
echo "Checking project directory..."
if [ -d ~/projects/aglc/aglc-crm ]; then
    echo -e "${GREEN}✓${NC} Project directory exists"
    cd ~/projects/aglc/aglc-crm

    if [ -f .env.local ]; then
        echo -e "${GREEN}✓${NC} Environment file exists"
    else
        echo -e "${YELLOW}⚠${NC} Environment file not found"
    fi

    if [ -d node_modules ]; then
        echo -e "${GREEN}✓${NC} Dependencies installed"
    else
        echo -e "${YELLOW}⚠${NC} Dependencies not installed"
    fi
else
    echo -e "${RED}✗${NC} Project directory not found"
fi

echo ""
echo "Checking services..."
if curl -s http://localhost:5173 > /dev/null; then
    echo -e "${GREEN}✓${NC} Frontend is running"
else
    echo -e "${RED}✗${NC} Frontend is not running"
fi

if curl -s http://localhost:3000 > /dev/null; then
    echo -e "${GREEN}✓${NC} Backend is running"
else
    echo -e "${RED}✗${NC} Backend is not running"
fi

if curl -s http://localhost:5050 > /dev/null; then
    echo -e "${GREEN}✓${NC} pgAdmin is running"
else
    echo -e "${RED}✗${NC} pgAdmin is not running"
fi

echo ""
echo "✅ Verification complete!"
```

```
EOF

chmod +x ~/verify_installation.sh
bash ~/verify_installation.sh
```

SUMMARY: DISTRIBUTION METHODS

Method	Pros	Cons	Best For
WhatsApp	Direct, familiar	Limited formatting	Team members
Telegram	Good formatting, bots	Requires account	Groups, channels
Email	Professional, attachments	Slower	Formal distribution
GitHub Gist	Version controlled, shareable	Requires GitHub	Developers
Slack	Team integration, searchable	Requires workspace	Organizations
One-Click Script	Fastest, automated	Requires trust	Technical users

---

FINAL DISTRIBUTION CHECKLIST

- [ ] Create deployment script (AGLC_CRM_DEPLOY.sh)
- [ ] Create quick reference card
- [ ] Create step-by-step commands
- [ ] Create email-friendly HTML guide
- [ ] Create GitHub Gist
- [ ] Create Slack message
- [ ] Create Telegram bot script
- [ ] Create verification script
- [ ] Test all scripts locally
- [ ] Share via WhatsApp
- [ ] Share via Telegram

- [ ] Share via Email
- [ ] Share via Slack
- [ ] Share via GitHub
- [ ] Collect feedback

Status: READY FOR DISTRIBUTION
All files are tested and production-ready
Choose your preferred distribution method and share with your team